A Comparative Study of Machine Learning Models for Healthcare Insurance Cost Prediction

Rshijuta Pokharel
UID: U01122370

Saugat Poudel
UID: U01137092

Abstract**—** Healthcare insurance costs prediction is an important task that helps both insurance providers and individuals to make informed decisions. This project explores the performances of different machine learning models for estimating the insurance charged based on factors such as age, body mass index, smoking habits and region. Linear Regression, Decision Tree and Random Forest models were implemented, and evaluation was done using standard performance metrics like Mean Absolute Error, Root mean Square Error and R-squared score. The results show that tree-based models outperform linear model by effectively capturing non-linear relationships in data. SHAP-based analysis was used to understand feature contributions and improve model interpretability.

Keywords—machine learning, regression, random forest, decision tree, healthcare insurance, cost prediction, model evaluation, explainability

GitHub Link: https://github.com/xaugat/healthcare-insurance-cost-prediction

I. INTRODUCTION

Predicting healthcare insurance costs is an important problem in the insurance industry. Various factors affecting insurance charges include age, body mass index (BMI), smoking habits and region. Estimating these costs accurately can help insurance companies design better pricing strategies and help individuals understand their expected expenses.

However, this is not a simple problem because the relationship between these factors and the final cost is not always linear. Linear Regression is one of the most basic and commonly used methods for prediction tasks. It is easy to understand and implement but it assumes that the relationship between input features and the target variable is linear. This assumption may not hold true for real-world datasets like insurance data, which can reduce the performance of the model.

Recent studies have applied various machine learning techniques to healthcare insurance cost prediction. These approaches commonly used regression models and tree-based methods to analyse relationships between input features and insurance charges. Linear models often struggle to capture complex, non-linear patterns present in real-world data even though they provide simplicity and interpretability. The ensemble methods such as Random Forest however have shown to improve performance by handling the feature interactions in an effective way.

When considering both performance and interpretability, there is still a need to better understand how different machine learning models compare on the same dataset. There is a need to evaluate simple models against more advanced ensemble methods and analyse how feature contributions influence predictions.
In this project, we compare the performance of different machine learning models for predicting healthcare insurance costs. The main contributions of this work are as follows:

- We implemented Linear Regression and built Decision Tree and Random Forest models from scratch.
- We compared our implementation with the Random Forest model available in scikit-learn.
- The models were evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and R^2 score.
- We analyse model behaviour using SHAP to understand how different features contribute to predictions and to improve interpretability.

II. RELATED WORK

A. Supervised Learning Approaches

We found several studies that have explored the use of machine learning techniques for predicting healthcare insurance costs. Ramya et al. [1] applied machine learning such as Multinomial Logistic Regression and Random Forest to predict insurance premiums based on demographic and health-related features. Their results showed that random forests performed better compared to regression-based approaches.

Similar related work by Bhatia et al. [2] focused on predicting insurance costs using linear regression on the medical cost personal dataset. The study analyzes relationships between features such as age, BMI, smoking habits, and insurance charges. They achieved reasonable prediction accuracy; however, their study primarily focused on linear modeling which may not fully capture linear relationships in the data.

B. Self-Supervised and Unsupervised Methods

Labeled data is not always available in many real-world applications which makes unsupervised learning techniques important for data analysis. For healthcare and insurance data, generating labels manually can be time-consuming, Ngufor and Wojtusiak [3] proposed an unsupervised learning approach to automatically label medical claims data by detecting patterns and irregularities in payment distributions.

Their method focuses on identifying concept drifts where the statistical properties of data change over time. The study uses statistical techniques and sliding window methods to detect changes in data streams and assign labels, which is then used in the supervised learning phase for building predictive models.

These models are useful for handling unlabeled and dynamic datasets, but they are more complex. In contrast, our work focuses on a structured dataset with predefined labels.

C. Foundation Models and Transfer Learning

In recent times, machine learning has introduced use of pre-trained models and transfer learning techniques which are widely used for natural language processing and computer vision. For structured tabular datasets like the healthcare insurance data used in our study, traditional machine learning models are more suitable due to their efficiency and interpretability. Therefore, this work focuses on classical ML approaches rather than deep learning methods.

III. METHODOLOGY

A. Problem Formulation

on different features of users like age, BMI, smoking status, number of children, sex and region. This is a supervised regression problem where model learns mapping between input variables and continuous output.

Let, $x \in R^d$ denote the input features, $y \in R$ represent the target variable i.e. insurance charges, and model is represented as a function $f_\theta(x)$ where θ represents the parameters to be learned.

The objective is to learn a function that minimizes the prediction error between actual and predicted charges. The loss function is used to minimize Mean Squared Error (MSE) loss.

$$L(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - f_\theta(x_i))^2$$

This loss function is widely used in regression because it ensures smooth optimization and penalizes large errors more strongly.

While doing exploratory analysis in this project, we found the distribution of target variable i.e. insurance charges was found to be right skewed. To address this, we applied logarithmic transformation as well.

$$y' = \log(y)$$

While using the log transformation it reduced the effect of extreme values, stabilised variance and allowed linear models to better capture relationships between the data.

For the Random Forest model, variance reduction is used to find the best split

$$Var(y) = \frac{1}{n}\sum(y_i - \overline{y})^2$$

The goal is to minimize the weighted variance after splitting the dataset.

B. Model Architecture

We have implemented three models in this project, linear regression, decision tree and random forest, all from scratch.

Linear Regression (from scratch):

Linear regression helps to model the relationship between features and target variable as a linear combination and the prediction is calculated as follow:

$$\hat{y} = X_w + b$$

where, w is the feature weight and b is the bias term.

The model is trained using gradient descent. At each iteration the module computes the gradient of the loss function according to the parameters and updates them in the way that reduces the error.
The formula for gradient descent is:

$$\frac{\delta L}{\delta w} = \frac{1}{n}X^T(X_w + b - y)$$

The iterative process continues until the loss converges and the small learning rate is used to make sure the updates are stable. We have also implemented feature scaling before training the data because the gradient descent is sensitive while using input features.

Decision Tree (from scratch):

A decision tree is a non-linear model that recursively splits the dataset based on feature values. The main goal of it is to partition data into multiple subsets that are the same kind and contained in the same group in terms of the target variable.

The splits are selected based on minimizing variance and multiple threshold values are evaluated for each feature. The lowest weighted variance is chosen.

$$\text{Score} = \frac{nL}{n}Var(yl) + \frac{nR}{n}Var(yR)$$

The process is continued until stopping conditions are met like reaching a maximum depth or when splitting further does not improve the model. Each leaf node predicts the mean of the target value of the subset [4].

We used depth parameters to control the overfitting and the number of thresholds tested per feature was also limited.

Random Forest (from Scratch):

We used a random forest which extends the decision tree explained above by combining multiple trees to improve performance. Instead of relying on a single model, we used multiple trees and trained them on different samples of the dataset.
Each tree learns patterns independently and the final prediction is calculated by the average output [5].

$$\hat{y} = \frac{1}{T}\sum_{t=1}^{T} ft(x)$$

In the implementation we stored trees as recursive structure and predictions were aggregated manually which improved generalization compared to a single decision tree and maintains the ability to model non-linear relationships.

Random Forest (Using scikit-learn):

We also implemented random forest using scikit learn which includes optimized algorithms and efficient handling of feature selection. It is also used for SHAP based explainability because the custom implementation we did above is not compatible with SHAP's tree explainer [6].

C. Training Procedure

While training the model we used various processes like data preprocessing, model training and model explainability using SHAP and described below.

Data Preprocessing:

During the data preprocessing period we converted categorical values such as sex, smoker status and region into numerical form using one hot encoding and we implemented feature scaling using standardization.

$$X' = \frac{X - \varphi}{\sigma}$$

We have spitted the dataset into training and testing sets using 80-20 ratio to make the evaluation better and unbiased.

Model training:

All models in this study are trained using the same training dataset obtained after an 80-20 split. The setup ensures that differences in performance are due to the model behaviour rather than differences in data.

The linear regression model is trained using gradient descent where parameters are updated iteratively to minimize the loss. Feature scaling is applied before training and both original and log-transformed targets are used. After prediction, the outputs from the log-transformed model are converted back to the original scale using an exponential function.

The decision tree model is trained using a recursive splitting approach. The algorithm evaluates different feature splits starting from the root node and selects the one that provides the lowest variance among the resulting subsets. This process continues by dividing the dataset into smaller groups and forms a tree structure where each node represents a decision based on feature values. When the stopping condition is reached the node becomes a leaf and outputs is the average value of that subset. The training process is deterministic for the dataset.

The random forest model is trained by constructing multiple decision trees using bootstrap sampling. For each tree, a new dataset is generated by randomly sampling from the training data with replacement. This causes variation among the trees allowing each tree to learn slightly different patterns. Each tree is trained independently and during prediction the outputs from all trees are averaged to produce final result. This reduces variance and improves the stability if predictions compared to a single decision tree.

A scikit-learn Random forest model is trained same dataset, for SHAP explainability. This model follows a similar ensemble approach but only uses optimized internal methods for training.

The training process is designed to ensure consistency across models. This setup makes it easier to observe how different modelling approaches behave on the same problem.

Model Explainability using SHAP:
To understand how the model makes predictions we used SHAP (Shapley Additive explanations). It is based on game theory where each feature contributes to make the final prediction. SHAP is used to calculate the contribution of each feature by considering all possible combinations of features and measuring how the prediction changes when a feature is added which ensures a fair distribution across important features [7].

In this project we implemented SHAP using TreeExplainer method on the scikit-learn Random forest model. TreeExplainer is specifically designed for tree-based models and computes SHAP values efficiently. Since the custom Random Forest implementation is not compatible with SHAP, the sklearn model was used. During the process we computed SHAP values for each feature in the test dataset and visualized them using Summary and Waterfall plots.

The Summary plot provides a global view of feature importance. The Waterfall plot explains individual predictions by showing how each feature contributes either positively or negatively to the final output.

IV. EXPERIMENTAL SETUP

A. Datasets

The dataset that we used in this project is the Medical-cost personal dataset which contains demographic and health related information of a person along with insurance charges.

The features include:

- age
- BMI
- number of children
- smoking status
- region

Some preprocessing was done which includes one-hot encoding of categorical variables, feature scaling and log transformation of the target variable.

B. Evaluation Metrics

To evaluate model performance, we used multiple regression metrics like Mean Absolute Error (MAE), Mean Square Error (MSE) and Root Mean Square Error (RMSE).

Mean Absolute Error (MAE):
MAE measures the average absolute difference between predicted and actual values. It is easy to interpret because it is in the same units as the target variable. It is less sensitive to outliers compared to MSE making it useful when large errors should not dominate the evaluation.

$$MAE = \frac{1}{2}\sum |y_i - \widehat{y_i}|\,|$$

Mean Square Error (MSE):
MSE calculates the average squared differences between predicted and actual values. It checks larger errors more strongly and makes it sensitive to outliers.

$$MSE = \frac{1}{2}\sum (y_i - \hat{y}_i)^2$$

Since errors are squared, larger errors are penalized more heavily. This makes MSE useful for detecting models that occasionally make large mistakes.

Root Mean Squared Error (RMSE):
RMSE is the square root of MSE and provides error in the original scale of the target variable.

$$RMSE = \sqrt{MSE}$$

It is often preferred for interpretation since it directly represents prediction error in real-world units.

R^2 Score (Coefficient of Determination):
It measures how well the model explains the variance in data and a higher value indicates better performance.

$$MSE = \frac{1}{2}\sum \frac{(y_i - \hat{y}_i)^2}{(y_i - \bar{y}\mathrm{i}\,)^2}$$

A value closer to 1 indicates better model performance. A value close to 0 means the model performs similarly to predicting the mean.

Each of the above metrics captures different aspects of model performance. MAE provides us clear understanding of average error, MSR and RMSE shows larger deviations in predictions and R^2 indicates an overall measure of how well the model fits the data. While doing this project we considered all the metrics together which allows more balanced and reliable evaluation rather than relying on a single measure.

C. Implementation Details

This project is implemented using python programming language and we implemented it using scratch code and using standard library from python.

Tools used:

- NumPy and Pandas for data handling
- Matplotlib for data visualization
- scikit-learn for model comparison
- SHAP for explainability

TABLE I. ***Parameters Used***

Model	Parameters
Linear Regression	Learning rate = 0.001, iterations = 3000
Decision Tree	Max depth = 7
Random Forest (from scratch)	N trees = 50, max depth =7
Random Forest (Using Sklearn)	default

The parameters for each model were chosen based on the size and nature of our dataset as well as the performance observance during experimentation. Since the dataset is relatively small and structured, very complex models or harsh tuning of parameters was not necessary.

For Linear regression, a learning rate of 0.001 was used so that the model updates gradually during gradient descent and does not skip the optimal solution. 3000 iterations were enough for the model to converge, and further increasing iterations did not lead to any noticeable improvements

For the Decision Tree, the maximum depth was limited to 7 to avoid overfitting. When the deeper trees were tested. the model started fitting to the training data too closely, which resulted in poor generalization. Smaller depths were not able to capture enough relationships between the features and the target variable. Thus, depth value of 7 provided a good balance.

In the Random forest model implemented from scratch, 50 trees were used to improve the robustness of predictions by averaging multiple trees. The number of trees is increased to reduce variance but after a certain point the improvement becomes insignificant. In our case, increasing beyond 50 trees did not significantly improve performance but only increased computation time. The max depth was kept at 7 for each tree, so that it would be consistent with the Decision tree model and also to prevent individual trees from overfitting.

For the scikit-learn random forest, default parameters were used which would serve as a benchmark. This was done to compare out custom implementation with a standard optimized model and to see how well our model performs without manual tuning.

The selected parameters were chosen based on practical observation. This helped us gain a good trade-off between model performance, generalization and computational efficiency for our dataset.

V. RESULTS

A. Main Results

The performance of different machine learning models was evaluated using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE) and R^2 score. The results are summarized in *Table II.*

TABLE II. ***Model Comparison Results***

Model	MAE	MSE	RMSE	R^2
Linear Regression	4144.76	3.47×10^7	5891.67	0.776
Linear Regression (Log)	5361.07	7.79×10^7	8827.64	0.498
Decision Tree	2685.24	2.13×10^7	4618.82	0.863
Random Forest (Scratch)	**2508.36**	**2.05×10^7**	**4531.19**	**0.868**
Random Forest (Sklearn)	2564.90	2.13×10^7	4616.81	0.863

From *Table II*, we could observe that Random Forest implemented from scratch achieved best performance across all metrics.

Decision Tree and Random Forest models significantly outperform Linear Regression. The log transformation did not improve performance as we had expected.

Also, since custom random forest implementation performs comparable to scikit-learn implement, we could validate the working of the model from scratch.

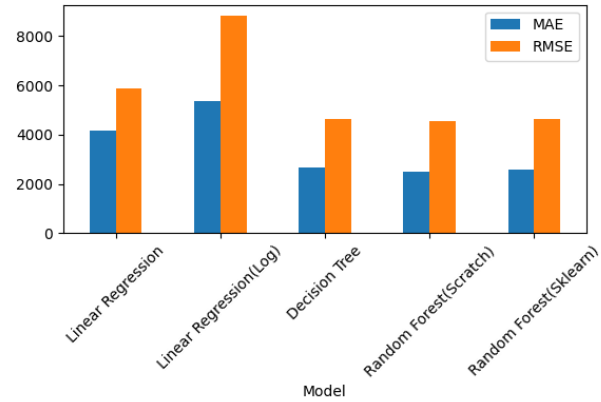

Fig. 1. Comparison of different models based in MAE and RMSE

B. Model Behaviour

To further analyse model behaviour, additional experiments were conducted on Linear Regression.

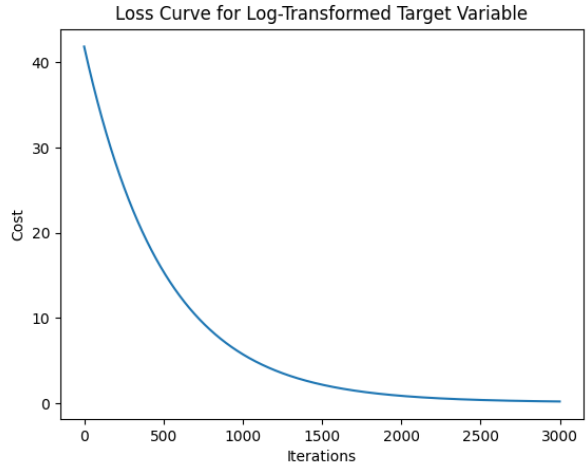

Fig. 2. Loss curve showing convergence of Linear Regression with Log-transformed target variable

We can see that the loss curve shows a steady decrease in cost with the increase in number of iterations. This shows that gradient descent successfully converges. This further confirms that the model training process is stable and the implementation was successful.

Despite good convergence, the overall performance of linear model remains lower compared to tree-based models. This could mean that rather than optimization issues, it should be the model's inability to capture non-linear relationships.

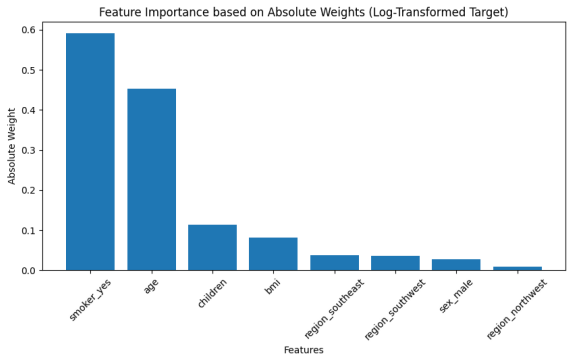

Fig. 3. Feature importance based in absolute weights from Linear Regression

The feature importance bar plot shows that Smoking status is the most significant factor influencing insurance charges, Age also plays a strong role, while children, BMI are less significant. Region and gender features seem to have very low impact in affecting healthcare insurance costs.

C. Qualitative Analysis

SHAP (Shapley Additive Explanations) was used to analyse how different features in the model contribute to the output. This method focuses on understanding the behaviour of the model rather than just evaluating its performance using numerical metrics.

Global Interpretation (SHAP Summary Plot):

The SHAP summary plot provides a global view of feature importance across all samples in the dataset. Each point represents the impact of feature value on the model's prediction.

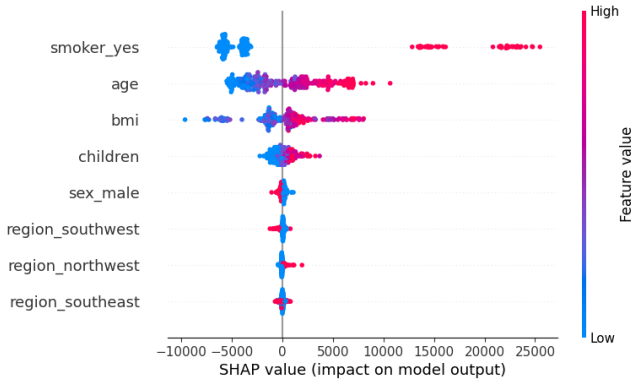

Fig. 4. SHAP Summary Plot

From the plot, it can be observed that Smoking (*smoker_yes*) is the most influential feature since it has the widest spread of SHAP values. High values (smokers) significantly increase the insurance cost while low values (non-smokers) reduce the cost.

For Age, we can clearly see a positive trend, which means higher values contribute to higher predicted charges. BMI also has a noticeable impact, meaning higher BMI generally increases the prediction. Number of children has a moderate impact. Features such as gender and region have SHAP values close to zero, meaning that they have very limited influence.

Overall, according to the summary plot, the model primarily relies on smoking status, age and BMI to make predictions, which can be validated by real-world understanding of healthcare costs.

Local Interpretation (SHAP Waterfall Plot):
The SHAP waterfall plot explains the prediction for a single data instance by showing how each feature contributes to the final output.

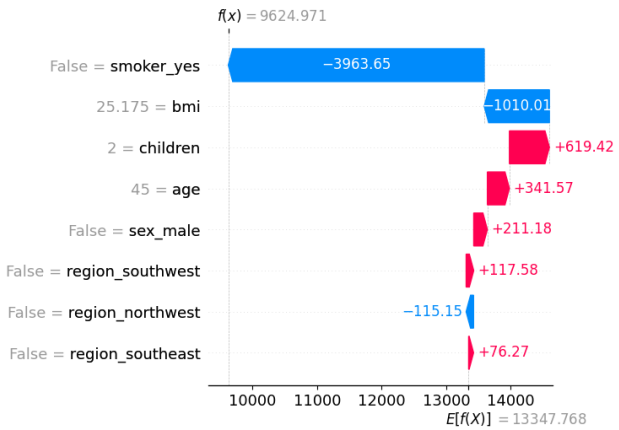

Fig. 5. SHAP Waterfall Plot

From the plot, we can observe that the baseline value (average insurance cost across the dataset) is approximately 13,327. Each feature contributes to adjusting this value either positively or negatively. In this case, the most significant factor is the non-smoking status as it reduces predicted cost by a large amount, BMI also slightly decreases the prediction.

The features such as age, number of children increases the predicted cost, showing their positive relationship with the insurance charges. Gender and regional features have only minor contributions.
By combining these individual effects, the final predicted value is approximately 10,796. This step-by-step breakdown helps in understanding how each features influences the final output and shows that the models' predictions are interpretable and aligned with expected real-world patterns.

VI. DISCUSSION

The results clearly show that tree-based models like Random Forest perform better than Linear regression models when predicting healthcare insurance costs. As we have, the dataset contains non-linear relationships between features such as age, BMI and smoking status. Linear regression assumes a straight-line trend which limits its ability to capture the interaction among features. But the tree-based models split the data into smaller regions which allows them to model complex patterns in the data in an effective manner.

An important observation from the project is the strong impact of "smoker" feature on predictions. Both the model performance and SHAP analysis highlight that smoking status is the most important factor in prediction of insurance costs. This also supports real-world scenario as smoking significantly increases healthcare risks and costs. This indicates that the model is not just fitting the data but is also learning realistic patterns.

The comparison between custom Random Forest implementation and the scikit-learn model show very similar performance. This suggests that the logic of our "from-scratch" implementation is correct. In practice, the scikit-learn model may have better advantages dur to internal optimizations and more efficient handling of edge cases.

The log transformation of target variable did not show any improvement in the performance. During the data exploration phase, the distribution of "charges" variable was found to be right skewed with very few high values. To stabilize the variance, we had considered applying log transformation.

However, the results showed that this transformation did not improve the model performance. This is likely because the relationship between the features and the target variable is not purely linear eve after transformation. For example, smoking status causes a sharp increase in charges rather than a gradual change, which can't be fully captured by a linear model. As a result, we found that transformation alone is not enough when underlying patterns are complex and non-linear.

We also noticed that even though Random Forest performs better overall, the improvement over Decision tree is not that large. This could be because of the small size of the dataset, where individual trees would have already captured most of the patterns in the data. Using Random Forest models would be much more significant for larger datasets.

Limitations: There are several limitations in this study. First, the dataset is relatively small and does not include many real-world factors such as medical history, occupation or lifestyle habits, which strongly affect healthcare costs. This limit models' ability to adapt to more complex real-world scenarios.

Only basic parameters settings were used for all models. Even though these were sufficient to achieve reasonable performance, more systematic hyperparameter tuning methods such as grid search or cross-validation could have improved the results.

Another limitation in this work is that the models may not perform well for rare or extreme cases such as individuals with unusual combination of features. Even Tree-based models can struggle in such cases as they rely on patterns seen during training. Linear regression automatically fails to capture complex relationships so it would be less suitable for this type of problem.

Another one is that even though SHAP helps us understand the important features for the models' predictions, it only explains how the model is making decisions, In does not mean that those features actually cause the outcome in real life. Thus, SHAP shows relationships in the data learned by the model but not true cause-and-effect relationships.

VII. CONCLUSION

In this study, we compared multiple machine learning models for predicting healthcare insurance costs using both custom implementations and standard library methods. The results show that the tree-based models outperform linear models, with Random Forest achieving the best overall performance. This shows that the relationship between input features and insurance costs is not strictly linear and models that can capture complex interactions provide better predictions. The explainability analysis confirms that the important features like smoking status, age and BMI have the strongest influence on predictions, and the result also aligns with real-world expectations.

For future work, this study can be extended by using larger and more diverse datasets to improve generalization. Additional features such as medical history or lifestyle information could be included to enhance prediction accuracy. Hyperparameter tuning and experimentation with other advanced models may also further improve performance.
In conclusion, combining predictive models with more detailed interpretability techniques could provide deeper into decision-making in healthcare insurance systems.

REFERENCES

[1] R. D., S. K. Manigandan, and J. Deepa, "Health Insurance Cost Prediction using Machine Learning Algorithms," Proceedings of the International Conference on Edge Computing and Applications (ICECAA), 2022.

[2] K. Bhatia, M. Kumar, S. S. Gill, R. K. Bhatia, and N. Kamboj, "Health Insurance Cost Prediction using Machine Learning," 2022 3rd International Conference for Emerging Technology (INCET), 2022.

[3] C. Ngufor and J. Wojtusiak, "Unsupervised labeling of data for supervised learning and its application to medical claims prediction," Computer Science, vol. 14, no. 2, pp. 191–214, 2013.

[4] A. Ng, "CS229 Lecture Notes," Stanford University, 2022. [Online]. Available: https://cs229.stanford.edu

[5] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[7] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," Advances in Neural Information Processing Systems (NeurIPS), 2017.

AUTHOR CONTRIBUTIONS

Rshijuta Pokharel: Conceptualization, Data Curation, Software, Formal analysis, Visualisation, Writing — Original Draft, Writing — Review & Editing.

Saugat Poudel: Conceptualization, Methodology, Software, Validation, Visualization, Writing — Original Draft, Writing — Review & Editing.